

FT Operations Portal — Frontend Design System, UI Architecture & UX Transformation Study

Project	FT Operations Portal (NAFFCO Operations Platform)
Report type	Design System / UI Architecture / UX / Frontend Codebase Transformation Study
Date	22 June 2026
Audited branch	<code>docs/frontend-design-ux-transformation-study</code> (cut from <code>main</code>)
Audited main SHA	<code>1ed145a</code> (Merge PR #132 — Step 19.5A Procurement Core UX)
Prepared by	Claude (acting as Senior Software Architect / Enterprise UX Strategist / Design System Architect)
Scope	Frontend design system, UX, UI architecture, component governance, data-loading strategy, and an A-to-Z transformation roadmap
Status	Analysis & planning only — no source code, routes, business logic, DB, RLS, or Supabase queries were changed

1. Cover & Scope Statement

This is a **read-only architectural and UX study**. It evaluates the entire `src/` frontend of the FT Operations Portal as it stands at main SHA `1ed145a`, and proposes a practical, staged path to a world-class, enterprise-grade operations platform.

What this report is: a grounded diagnosis (every major claim is backed by a repository measurement), a UI-library strategy decision, a future design-system blueprint, and a phase-by-phase, PR-by-PR execution plan.

What this report is not: an implementation. No component was refactored, no route changed, no query touched. All recommendations are explicitly sequenced so that business-critical workflow gates (SO approval, quotation conversion, PO approval, WO/PN gates, store custody/serials, QC release, AFS readiness, role access) are protected.

Evidence basis (measured this session):

- 124 page files in `src/pages`, totalling ~41,938 lines.
- 49 reusable components across 9 component folders.
- 125 routes, 124 lazy-loaded; 10 roles in `roleMatrix.ts`.
- 103 of 124 pages fetch data through hand-written `useEffect`; 112 pages reference `supabase` directly.
- 0 pages use `react-hook-form` or `zod` (both are installed dependencies).
- 0 unit tests; 1 Playwright screenshot spec.
- ESLint baseline: 74 problems (38 errors, 36 warnings).

2. Executive Summary

2.1 Current state assessment

The portal is a **mature, functionally complete, role-driven operations system** with real governance depth (SO approval/routing, quotation→SO conversion, high-value PO approval, WO/PN execution gates, store custody & serial tracking, QC NCR/release, AFS readiness). The engineering fundamentals are better than typical internal tools: routes are lazy-loaded, a design-token foundation exists (Step 19.1), and a structured UX modernization program is already underway (Steps 19.1 → 19.5A merged).

However, the frontend sits at an **inflection point**. Three structural issues now limit both velocity and polish:

- A half-migrated design system.** Two parallel component layers coexist — hand-rolled `ui/*` components (e.g. `Button.tsx`, `Card.tsx`, `Badge.tsx`, `PageHeader.tsx`) and shadcn-style `ui/primitives/*` + `common/*` — with **incompatible APIs** (the custom `Button` takes `variant="primary"`; the primitive `button` takes `variant="default"`). The design-token work from Step 19.1 is real but **only partially propagated**: 77 of 124 pages still use the legacy `rounded-xl` radius and 48 still use the legacy `font-semibold text-gray-700` table header style.
- No data-loading architecture.** 103 of 124 pages fetch via manual `useEffect` + `useState`, with no caching, no shared query layer, no pagination, and no request deduplication. This is the single largest source of the recurring `react-hooks/set-state-in-effect` lint failures (17 instances) and of perceived sluggishness.
- Near-zero automated safety net.** 0 unit tests and a single screenshot spec mean every refactor is validated only by `tsc` + `eslint` + manual review. For a system with this many business gates, that is the central risk to any transformation.

2.2 Biggest problems

#	PROBLEM	EVIDENCE
P1	Duplicate component layers with divergent APIs	<code>Button.tsx</code> vs <code>primitives/button.tsx</code> ; <code>Card</code> , <code>Badge</code> , <code>PageHeader</code> , <code>EmptyState</code> all duplicated
P2	Design tokens not fully propagated	77/124 pages on <code>rounded-xl</code> ; 48/124 legacy table headers
P3	No query/cache layer	103/124 pages use manual <code>useEffect</code> ; React Query/SWR absent
P4	No test coverage	0 unit tests; 1 Playwright spec
P5	Installed-but-unused form stack	<code>react-hook-form</code> + <code>zod</code> present, 0 page usage — forms are hand-rolled and validated ad hoc
P6	Mega-pages	<code>ProjectDetail.tsx</code> 2,081 lines; 8 pages > 700 lines
P7	Dual color semantics	shadcn <code>--primary</code> is navy; custom components use <code>brand</code> red — "primary" means two different colors
P8	Lint debt	74 problems (38 errors), incl. 23 <code>no-explicit-any</code> , 17 <code>set-state-in-effect</code>

2.3 Biggest opportunities

- Finish what is already working.** The Step 19.1 → 19.5A pattern (skeletons, `rounded-lg`, uppercase tracked table headers, `tabular-nums`, `bg-white` inputs) is proven and low-risk. Rolling it across the remaining ~7 modules is high-value and safe.

- **Consolidate to one component layer.** Standardizing on the shadcn/Radix + Tailwind foundation (already in the dependency tree) eliminates the duplicate-API tax permanently.
- **Introduce a query layer once, benefit everywhere.** A single React Query adoption removes the entire `set-state-in-effect` class of bugs and unlocks caching, pagination, and background refresh.

2.4 Recommended transformation direction

Evolve, do not rewrite. Standardize on **Option C — a formal internal NAFFCO design system built on Radix + Tailwind + CVA** (the stack already present), consolidate the duplicate component layers into it, finish the module-by-module UX rollout, then layer in React Query and a test harness. **Do not migrate to MUI/Ant/Mantine** — the cost is enormous and the current stack can reach world-class quality.

2.5 Expected benefits

- Consistent, premium, enterprise visual identity across all 124 pages.
- ~30-40% less UI code through component consolidation.
- Elimination of the `set-state-in-effect` bug class and faster perceived performance via caching.
- A safety net (tests + Storybook) that makes future change cheap and low-risk.

2.6 Recommended priority sequence

1. **Phase 0** — stabilize: fix the 74 lint problems, freeze a token contract.
2. **Phase 1** — finish module UX rollout (Store, Factory, QC, AFS, After-Sales, Reports, Admin).
3. **Phase 2** — consolidate duplicate components into one design system.
4. **Phase 4** — introduce React Query (note: sequence-numbered to match \$16).
5. **Phase 3 / 5 / 6 / 7 / 8** — page-architecture refactor, performance, dashboards, accessibility, testing.

3. Current System Overview

3.1 Frontend stack

LAYER	TECHNOLOGY	VERSION
Framework	React	18.3.1
Build	Vite	6.0.5
Language	TypeScript	5.7.2
Routing	react-router-dom	6.28.0
Styling	Tailwind CSS	3.4.17
Primitives	Radix UI (11 packages)	1.x-2.x
Variants	class-variance-authority	0.7.1
Icons	lucide-react	0.469.0
Data	@supabase/supabase-js	2.106.2
Tables	@tanstack/react-table	8.21.3 (used in 1 place)
Forms	react-hook-form 7.79 + zod 4.4	installed, unused in pages
E2E	@playwright/test	1.61 (1 screenshot spec)
Server cache	—	none (no React Query / SWR)

3.2 Routing model

`src/app/App.tsx` (320 lines) defines **125 routes**, of which **124 are lazy-loaded** — code-splitting is already in place and is a genuine strength. Route protection is layered through three guards in `src/components/auth/`: `ProtectedRoute.tsx`, `RequireRole.tsx`, and `PermissionGate.tsx`.

3.3 Role-based UX model

Ten roles are defined in `src/lib/roleMatrix.ts` (181 lines): `admin`, `operations_manager`, `sales_user`, `sales_coordinator`, `procurement_user`, `factory_user`, `store_user`, `qc_user`, `afs_user`, `viewer`. Navigation is data-driven from `src/data/navigation.ts` (1,089 lines, 145 labelled items) and filtered per role. Auth state flows from a single `src/context/AuthContext.tsx` consumed via `useAuth` and `usePermission` hooks.

3.4 Major modules (page inventory)

MODULE	PAGES	UX-UPGRADED?
Reports	15	✗ Not yet
Store / Warehouse	12	✗ Not yet
Projects / Sales Orders	11	✓ 19.4A (detail/list); 19.4B pending merge
Procurement	11	✓ 19.5A
Factory / Production	8	✗ Not yet
Admin	8	✗ Not yet
After Sales	4	✗ Not yet
Quotations / Commercial	3	✓ 19.3
QC	3	✗ Not yet
AFS / Dubai	6	✗ Not yet
Sales	2	✓ 19.3
Coordinator	1	✓ 19.3
Control Tower	1	✗ Not yet
Dashboard / Settings / Inbox / WoPn / Notifications	misc	partial

3.5 Current design-system approach

A token foundation was established in Step 19.1 (`tailwind.config.js` + `src/styles/index.css`): a `brand` (NAFFCO red) and `charcoal` scale, shadcn CSS-variable tokens, a 5-stop shadow scale (`--shadow-xs ... xl`), surface hierarchy (`--surface-1/2/3`), and `--radius: 0.375rem` . This is solid. The problem is **propagation and duplication**, not the tokens themselves.

3.6 Component organization

49 components across: `ui/` (30), `common/` (8), `data-display/` (3), `feedback/` (3), `status/` (3), `auth/` (3), `documents/` (2), `features/` (2), `layout/` (2). The `ui/` folder mixes three generations: hand-rolled capitalised components (`Button.tsx`), shadcn lowercase primitives (`input.tsx` , `select.tsx` , `table.tsx`), and a `primitives/` sub-layer.

3.7 Data-fetching approach

Each page owns its fetching: an `async` IIFE inside `useEffect` , a mock-vs-Supabase branch keyed on `isSupabaseConfigured` , and local `useState` for `data` / `loading` / `error` . There is no shared client wrapper beyond `src/lib/supabase.ts` and per-domain audit helpers (`projectAudit.ts` , `procurementAudit.ts` , etc.).

3.8 Known recent improvements (merged to main)

- **19.1** Design system foundation & app shell (PR #126).
- **19.2** Functional pages UX roadmap doc (PR #127).
- **19.3** Commercial / Sales Coordinator UX (PR #128).
- **19.4A** Projects / Sales Orders UX (PR #130).
- **19.5A** Procurement Core UX (PR #132).
- **19.4B** Project Creation Wizard UX — **PR #131 open, not yet in main** (targets the modernization branch).

4. Design System Diagnosis

4.1 Tailwind tokens

Strong. `brand` and `charcoal` scales are complete (50–950), the shadow scale is variable-driven, and `borderRadius` maps `lg/md/sm` to `--radius` . **Gap**: there is no semantic spacing scale or typographic scale beyond Tailwind defaults, and status colors are applied as raw utility strings (`bg-amber-50 text-amber-700`) rather than tokens.

4.2 CSS variables

Well-structured in `index.css` . **Risk**: `--primary: 222 47% 11%` (dark navy, shadcn default) is **semantically wrong** for this product — "primary" in NAFFCO terms is brand red (`--ring` and `--destructive` are correctly red). Custom components hard-code `brand-600` ; primitive components reference `--primary` (navy). The word "primary" therefore resolves to two different colors depending on which component layer you land on.

4.3 shadcn primitives vs custom components — the core duplication

CONCEPT	CUSTOM LAYER	SHADCN/PRIMITIVE LAYER	API CONFLICT
Button	<code>ui/Button.tsx</code> (94 page imports)	<code>ui/primitives/button.tsx</code> (5 imports)	<code>variant="primary"</code> vs <code>variant="default"</code>
Card	<code>ui/Card.tsx</code>	<code>ui/primitives/card.tsx</code>	padding prop vs slot composition
Badge	<code>ui/Badge.tsx</code>	<code>ui/primitives/badge.tsx</code>	6 named variants vs CVA variants
PageHeader	<code>ui/PageHeader.tsx</code> (15 imports)	<code>common/page-header.tsx</code> (105 imports)	<code>action</code> vs <code>actions</code> prop
EmptyState	<code>ui/EmptyState.tsx</code> (51 imports)	<code>feedback/empty-state.tsx</code> (0 imports)	divergent props
Loading	<code>ui/PageLoader.tsx</code> (36), <code>ui/skeleton.tsx</code> (12)	<code>feedback/loading-state.tsx</code>	spinner vs skeleton

This is the **single highest-leverage cleanup** in the codebase: it touches consistency, onboarding, and every future UX PR.

4.4 Visual consistency

Improving but uneven. Quantified drift: **77/124 pages** still use `rounded-xl` (pre-19.1 radius) and **48/124** still use the legacy `font-semibold text-gray-700` table header. The 5 upgraded modules demonstrate the target state; the rest lag.

4.5 Status colors

Status is expressed through ad-hoc Tailwind pairs and through `Badge` variants (`neutral/warning/info/success/critical`). There is a `components/status/` folder (3 files) but status semantics are not centralised into a single source of truth, so the same business status can render with slightly different colors in different modules.

4.6 Typography / spacing / radius / shadows

Radius and shadows are tokenised and good. Typography relies on Inter + Tailwind defaults with no semantic scale (`text-display` , `text-h1` , ...). Spacing is raw Tailwind. This is acceptable short-term but should be formalised in the blueprint (§14).

4.7 Accessibility & contrast

Radix primitives bring solid keyboard/focus semantics for dialogs, selects, tabs, and tooltips. Risks: hand-rolled interactive elements (clickable table rows via `onClick` , custom file-upload labels) lack consistent focus-visible treatment; some muted-gray-on-off-white text pairings are borderline for WCAG AA. No automated ally checks exist.

4.8 Scalability of the current foundation

The foundation is scalable; the usage is not yet disciplined. Tokens + Radix + CVA is exactly the stack a world-class internal design system is built on. The blocker is governance: nothing prevents a new page from importing the legacy `Button` or hard-coding `rounded-xl` .

5. UI Library Strategy Assessment

Six options were evaluated against this specific codebase (124 pages, Radix already adopted, heavy Tailwind usage, deep role/governance logic).

Option A — Continue current Tailwind + hand-rolled components

- **Benefits:** zero migration; team already fluent.
- **Risks:** entrenches duplication; consistency stays manual; no governance.
- **Migration cost:** none. **Fit:** medium. **Maintainability:** low-medium. **Verdict:** insufficient on its own.

Option B — Standardize fully on shadcn/ui + Radix + Tailwind

- **Benefits:** uses what's installed; copy-in-repo ownership; excellent ally via Radix; low learning curve.
- **Risks:** shadcn is a pattern, not a governed system — without internal rules it drifts again.
- **Migration cost:** medium (consolidate duplicates, migrate `variant="primary"` → `default` call sites). **Fit:** high. **Verdict:** strong, and the technical substrate of the recommendation.

Option C — Formal internal NAFFCO design system on Radix + Tailwind + CVA ***(recommended)***

- **Benefits:** Option B **plus** governance — one tokenised, documented, Storybook-backed component library with semantic variants (`Button variant="primary|secondary|ghost|danger"` , status tokens, page templates). Premium brand identity becomes enforceable.
- **Risks:** requires upfront definition effort and discipline.
- **Migration cost:** medium (largely overlaps Option B work). **Fit:** highest. **Maintainability:** high. **Design quality ceiling:** highest. **Verdict: best long-term path; build it incrementally on top of the Option B consolidation.**

Option D — Migrate to MUI

- **Benefits:** batteries-included, mature ally, rich data grid.
- **Risks:** total restyle to escape "Material" look; emotion/styled runtime conflicts with the Tailwind mental model; **rewrite of all 124 pages**; brand expression harder.
- **Migration cost:** very high. **Fit:** low. **Verdict: not recommended.**

Option E — Migrate to Ant Design

- **Benefits:** dense enterprise components, strong tables/forms out of the box.
- **Risks:** opinionated design language fights NAFFCO brand; large bundle; less-Tailwind-aligned; full rewrite.
- **Migration cost:** very high. **Fit:** low-medium. **Verdict: not recommended.**

Option F — Migrate to Mantine (or similar)

- **Benefits:** modern DX, good hooks, good components.
- **Risks:** another full migration; parallel styling system; abandons sunk Radix/Tailwind investment.
- **Migration cost:** very high. **Fit:** low. **Verdict: not recommended.**

5.1 Scoring (see §6.A for the full matrix)

Option C scores highest on enterprise readiness, maintainability, design quality, accessibility, fit, and long-term scalability, while keeping risk and migration complexity moderate because it **reuses the existing stack**.

5.2 Final recommendation

- **Best option:** **C** (formal internal design system) implemented **via B** (consolidate onto shadcn/Radix/Tailwind).
 - **Why:** maximum design ceiling and maintainability at moderate, incremental cost; no rewrite; protects the running business system.
 - **What not to do:** do not adopt MUI/Ant/Mantine; do not run two component layers indefinitely; do not block module UX work waiting for the full system.
 - **What to defer:** Storybook and full token-package extraction can come after the duplicate layers are merged and the module rollout is complete.
-

6. Evaluation / Scoring Tables

6.A UI Library Strategy Score (1 = poor, 5 = excellent; Risk/Complexity: 5 = low risk / low complexity)

CRITERION	A: CURRENT	B: SHADCN	C: INTERNAL DS	D: MUI	E: ANT	F: MANTINE
Enterprise readiness	3	4	5	4	4	4
Speed of implementation	5	4	3	2	2	2
Maintainability	2	4	5	3	3	3
Design quality potential	3	4	5	3	3	4
Accessibility maturity	3	4	5	5	4	4
Risk level (5=low)	5	4	4	1	1	1
Migration complexity (5=low)	5	3	3	1	1	1
Fit with current codebase	3	5	5	1	2	1
Long-term scalability	2	4	5	4	4	4
Total (/45)	31	36	40	24	24	24

6.B Module UX Priority Score (1-5; higher = more urgent/important)

MODULE	USER IMPACT	OPS IMPORTANCE	CURRENT UX GAP	IMPL. RISK (5=LOW)	DEPENDENCY (5=LOW)	BUSINESS CRITICALITY	PRIORITY
Store / Warehouse	5	5	5	3	4	5	27
Reports	4	4	5	4	4	4	25
Factory / Production	5	5	4	3	3	5	25
QC / NCR / Release	5	5	4	2	3	5	24
AFS / Dubai	4	4	4	3	4	4	23
Control Tower	4	5	4	3	3	4	23
After Sales	3	3	4	4	4	3	21
Admin	3	3	3	4	4	4	21
Viewer / Management	3	3	3	5	5	3	22

6.C Technical Debt Severity Score (1-5; Value if fixed: higher = more value)

DEBT	SEVERITY	COMPLEXITY	RISK	VALUE IF FIXED
Duplicate component layers	5	3	3	5
No query/cache layer	4	4	3	5
Token propagation incomplete	3	2	2	4
No test coverage	5	4	2	5
Mega-pages (ProjectDetail)	4	4	4	4
Lint debt (74 problems)	3	2	2	4
Unused form stack (RHF/zod)	2	3	2	3
Dual color semantics	3	2	2	3

7. UX / IA Diagnosis

- Navigation density:** `navigation.ts` carries 145 labelled items across 10 roles. Role filtering keeps any one user's menu reasonable, but the data file (1,089 lines) is large and should be modularised per domain.
- Role-based landing clarity:** strong concept — role dashboards and work centers exist (Steps 18.7x). Landing pages vary in polish; the upgraded modules feel premium, others feel like admin tables.
- Page hierarchy:** `PageHeader` + breadcrumb is consistent where the `common/page-header` is used (105 pages); the 15 still on `ui/PageHeader` differ subtly.
- Action placement:** generally top-right primary action + inline row actions. Good. Some forms bury the submit area.
- Workflow clarity:** governance banners (PO > SAR 10,000, WO/PN gate notices) are a genuine strength and should become a reusable `GovernanceNotice` component.
- Dashboard usefulness:** KPI strips exist (Procurement, Control Tower) but mix raw counts with navigation; metric cards should be standardised.
- List/table usability:** the upgraded pattern (uppercase tracked headers, `tabular-nums`, skeleton loading) is excellent; 48 pages still lag.
- Form usability:** hand-rolled, inconsistent validation messaging; no shared field components.
- Empty/loading/error states:** three competing implementations; skeletons only on 12 pages, spinners (`PageLoader` / `Loader2`) on ~50.
- Module consistency:** the central IA issue — five modules modernised, the rest not.
- Mobile/responsive readiness:** Tailwind responsive classes are used, but dense tables and multi-column forms are not yet mobile-optimised; clickable rows have no mobile affordance.

8. Functional Module UX Diagnosis

Legend — Phase = recommended improvement phase (see §16).

Sales / Commercial (✓ 19.3)

- **Strengths:** modernised; quotation governance (R-001 gate) visible.
- **Weaknesses:** `QuotationDetail.tsx` is 941 lines; `QuotationNew.tsx` 738.
- **High-impact:** split detail tabs into sub-components.
- **Risks:** quotation conversion logic must not move. **Phase:** 3. **Value:** medium.

Sales Coordinator (✓ 19.3)

- **Strengths:** dedicated queue; modernised.
- **Weaknesses:** `CoordinatorQueue.tsx` 446 lines.
- **High-impact:** query-layer adoption for live queue. **Phase:** 4. **Value:** medium.

Projects / Sales Orders (✓ 19.4A; 19.4B pending)

- **Strengths:** strongest UX in the app; skeletons, tabular numerics, governance notices.
- **Weaknesses:** `ProjectDetail.tsx` is **2,081 lines** — the single riskiest page.
- **High-impact:** decompose `ProjectDetail` tabs; adopt query layer for tab data.
- **Risks:** SO approval/routing, WO/PN gate cards. **Phase:** 3 + 4. **Value:** very high.

Procurement (✓ 19.5A)

- **Strengths:** just modernised; skeletons, approval visibility, `tabular-nums`.
- **Weaknesses:** `ProcurementPODetail.tsx` 884 lines (6-tab page).
- **High-impact:** decompose PO detail tabs.
- **Risks:** high-value PO approval (SAR 10,000), status transitions. **Phase:** 3. **Value:** high.

Store / Warehouse (✗)

- **Strengths:** deep functionality (receiving, custody, serials, photos).
- **Weaknesses:** 12 pages, none modernised; legacy tables/loaders.
- **High-impact:** full module UX rollout (skeletons, headers, inputs).
- **Risks:** receiving/custody/serial integrity — visual only, protect writes. **Phase:** 1. **Value:** very high (top priority, score 27).

Factory / Production (✗)

- **Strengths:** workspace + raw-material request flows.
- **Weaknesses:** `FactoryProjectWorkspace.tsx` 714 lines; not modernised.
- **High-impact:** module UX rollout; later split workspace.
- **Risks:** WO execution gate. **Phase:** 1 (UX), 3 (split). **Value:** high.

QC / NCR / Release Notes (✗)

- **Strengths:** governance-critical release gate.
- **Weaknesses:** `QCWorkQueue.tsx` 558 lines; not modernised.
- **High-impact:** module UX rollout with careful gate protection.
- **Risks:** NCR/rework/release gate — highest workflow sensitivity. **Phase:** 1 (UX), validate heavily. **Value:** high.

AFS / Dubai (✗)

- **Strengths:** delivery-readiness + missing-item evidence (Phase 1A).
- **Weaknesses:** 6 pages, not modernised; some detail UI incomplete (deferred items).
- **High-impact:** module UX rollout; finish missing-item evidence UI.
- **Risks:** AFS readiness logic. **Phase:** 1. **Value:** medium-high.

After Sales (✗)

- **Strengths:** maintenance flow closure exists.
- **Weaknesses:** 4 pages, not modernised.
- **High-impact:** module UX rollout. **Phase:** 1. **Value:** medium.

Operations Control Tower (✗)

- **Strengths:** SLA/cross-module visibility.
- **Weaknesses:** `ControlTower.tsx` 616 lines; KPI cards not standardised.
- **High-impact:** standardise dashboard/KPI components. **Phase:** 6. **Value:** high.

Admin (✗)

- **Strengths:** approvals, users, settings present.
- **Weaknesses:** `AdminApprovals.tsx` 764, `AdminUsers.tsx` 449; not modernised.
- **High-impact:** module UX rollout; admin-only guard review. **Phase:** 1/2. **Value:** medium.

Viewer / Management (✗)

- **Strengths:** read-only model exists.
- **Weaknesses:** not modernised; lowest risk to change.
- **High-impact:** quick UX rollout (safe). **Phase:** 1. **Value:** medium (best risk/reward).

Reports (✗)

- **Strengths:** 15 report pages; print support in CSS.
- **Weaknesses:** largest unmodernised surface; heavy mock data; `ReportsStore.tsx` 766 lines.
- **High-impact:** standardise report/table/print templates; executive dashboards.
- **Risks:** report export logic. **Phase:** 6. **Value:** high.

9. Component Architecture Diagnosis

9.1 Duplicated components (consolidate)

`Button`, `Card`, `Badge`, `PageHeader`, `EmptyState`, and loading states each have 2-3 implementations. Target: **one** of each, in a governed `ui/` design-system layer.

9.2 API inconsistencies

- `Button`: `variant="primary"` (custom) vs `"default"` (CVA).
- `PageHeader`: `action` (singular) vs `actions` (plural).
- `EmptyState`: divergent prop names.

These must be reconciled with a **single canonical API** and a codemod/manual migration of call sites (94 Button imports, 105 page-header imports).

9.3 Reusable pattern gaps (create shared components)

- `GovernanceNotice` (the amber/sky banner pattern repeated across modules).
- `StatusBadge` driven by a central status→variant map.
- `DataTable` standard (headers, `tabular-nums`, skeleton, empty state) — partially exists in `data-display/` but underused.
- `FormField` / `FormSection` (label + control + help + error) to replace hand-rolled inputs.
- `KpiCard` / `MetricCard` standard (one implementation; `common/metric-card.tsx` is the seed).
- `TableSkeleton` / `DetailSkeleton` (the skeleton patterns now copy-pasted in upgraded pages).

9.4 Consolidation vs local

- Consolidate:** all of §9.1 + §9.3.
- Keep local:** genuinely page-specific composites (e.g. `WoPnGateCard`, `RoutingSummaryCard`, `ApprovePanel`) — these encode business logic and should stay near their page, composed from shared primitives.

9.5 Recommended component folder structure

```
src/components/
  ui/           # design-system primitives (one Button, Card, Badge, Input, Table...)
  patterns/     # composed cross-module patterns (GovernanceNotice, StatusBadge,
                # KpiCard, DataTable, FormField, EmptyState, skeletons)
  layout/       # shell: Sidebar, Header, AppLayout
  feature/<domain>/ # page-specific composites kept beside their domain
```

9.6 Governance rules

- Pages may import only from `ui/` and `patterns/` (never another page's internals).
- No raw `rounded-xl`, `border-gray-300`, or `font-semibold text-gray-700` table headers — lint-enforced.
- One canonical component per concept; deprecated layers physically deleted.
- Every shared component has a Storybook story and documented props.

10. Page Architecture Diagnosis

10.1 Largest / riskiest pages

PAGE	LINES	ACTION
<code>ProjectDetail.tsx</code>	2,081	Split (tabs → sub-components)
<code>ProjectNew.tsx</code>	1,083	Split wizard steps (19.4B in flight)
<code>QuotationDetail.tsx</code>	941	Split tabs
<code>WoPnGate.tsx</code>	900	Split, protect gate logic
<code>ProcurementPODetail.tsx</code>	884	Split 6 tabs
<code>ReportsStore.tsx</code>	766	Standardise via report template
<code>AdminApprovals.tsx</code>	764	Split
<code>QuotationNew.tsx</code>	738	Split steps
<code>FactoryProjectWorkspace.tsx</code>	714	Split

10.2 Pages that should be split

All > ~600 lines (the nine above plus `Settings` 695, `ProcurementSupplierDetail` 629, `ControlTower` 616). Splitting is **structural only** — extract tab bodies into co-located sub-components, leave logic intact.

10.3 Pages that should stay simple

List pages (`Projects`, `Quotations`, `ProcurementRequests`, etc.) — keep them as thin list + shared `DataTable`.

10.4 Pages needing layout redesign

Reports (15) and Control Tower — move to standardised report/dashboard templates.

10.5 Pages needing data-architecture changes

Every detail page with multiple tabs (`ProjectDetail`, `PODetail`, `QuotationDetail`, `FactoryWorkspace`) — these benefit most from a query layer with per-tab cached queries.

10.6 Pages needing schema/live-data work

Report pages still on mock data; AFS missing-item evidence detail (deferred). Out of scope here; flagged for backlog.

10.7 Pages that must not be touched casually

WoPnGate.tsx , ProjectDetail.tsx (approval/routing/gate cards), ProcurementP0Detail.tsx (PO approval), QC release pages, Store receiving/custody/serial pages. Any change here requires the safety checklist in §19.

11. Performance & Data Loading Diagnosis

- **Manual useEffect fetching:** 103/124 pages. Each re-fetches on mount with no cache → repeated network calls on every navigation.
 - **No React Query/SWR:** no dedup, no background refresh, no stale-while-revalidate.
 - **set-state-in-effect** : 17 lint instances are a direct symptom of the manual pattern (synchronous setLoading(true) in effect bodies). A query layer removes this class entirely.
 - **Loading UX:** skeletons on only 12 pages; spinners on ~50 (PageLoader: 36, Loader2: 17). Skeletons should be the default.
 - **Pagination:** absent — list pages fetch all rows then filter client-side (e.g. .order('created_at') with no range). Fine at current scale, a cliff as data grows.
 - **Large-page loading:** multi-tab detail pages fetch everything up front (e.g. ProjectDetail loads all tab data in one effect); should fetch per active tab.
 - **Reports/mock data:** several report pages render mock arrays — real queries + caching needed.
 - **Caching opportunities:** reference data (projects list, approved suppliers, PR list) is re-fetched in multiple forms — ideal for shared cached queries.
- Recommended performance roadmap:** (1) make skeletons the default loading state during the module rollout; (2) introduce React Query as a foundation (Phase 4); (3) migrate detail pages to per-tab cached queries; (4) add server-side pagination to list pages (Phase 5); (5) lazy-load heavy report/chart code.

12. Accessibility & Usability Review

- **Color contrast:** muted-gray text on off-white (text-gray-400/500 on --surface-1) is borderline AA; audit and bump to text-gray-600 where needed.
- **Focus states:** Radix components good; hand-rolled clickable rows/file-upload labels need consistent focus-visible rings.
- **Keyboard navigation:** clickable <tr onClick> rows are not keyboard-reachable — provide a real link/button per row.
- **Form labels/help text:** present but inconsistent; standardise via FormField .
- **Table readability:** upgraded pattern is strong; roll out everywhere.
- **Status badge clarity:** centralise the status→color map so the same status always reads the same.
- **Loading/empty/error states:** unify on skeleton + one EmptyState + one ErrorState .
- **Responsive/mobile:** dense tables overflow; multi-column forms cramp on mobile; add responsive table patterns.

Accessibility checklist (adopt): semantic landmarks; one <h1> per page; keyboard-reachable interactive elements; visible focus; labelled inputs; AA contrast; aria-live for async messages; tested with keyboard-only and screen reader on the top 10 pages.

13. Technical Debt Register

ID	AREA	ISSUE	SEVERITY	IMPACT	RECOMMENDED FIX	DEPENDENCY	EFFORT	PHASE
TD-01	Components	Duplicate Button/Card/Badge/PageHeader/EmptyState layers	High	Inconsistency, slow onboarding	Consolidate to one DS layer	Token freeze	L	2
TD-02	Data	No query/cache layer; 103 manual useEffect	High	Perf, bugs, dup code	Adopt React Query	Phase 1 done	L	4
TD-03	Tokens	rounded-xl in 77 pages; legacy headers in 48	Med	Visual drift	Finish module rollout + lint rule	—	M	1
TD-04	Testing	0 unit tests, 1 e2e spec	High	Refactor risk	Vitest + RTL + Playwright smoke	—	L	8
TD-05	Pages	Mega-pages (ProjectDetail 2,081)	High	Maintainability	Split tabs into sub-components	DS layer	L	3
TD-06	Lint	74 problems (38 errors)	Med	CI noise, latent bugs	Burn down by rule	—	M	0
TD-07	Forms	RHF + zod unused; hand-rolled validation	Med	Inconsistent UX	Standardise FormField on RHF+zod	DS layer	M	3
TD-08	Color	--primary navy vs brand red	Med	Semantic confusion	Define semantic color tokens	Token freeze	S	2
TD-09	Data	No pagination on lists	Med	Scale cliff	Server-side ranges	React Query	M	5
TD-10	Nav	navigation.ts 1,089 lines monolith	Low	Maintainability	Split per domain	—	S	2
TD-11	Status	No central status→color map	Med	Inconsistent badges	StatusBadge + map	DS layer	S	2
TD-12	Loading	3 loading systems; spinners dominate	Med	Inconsistent UX	Skeleton-first, one system	DS layer	M	1/2

Effort: S < 1 wk · M 1–2 wk · L 3+ wk (one engineer, indicative).

14. Design Transformation Principles

1. **Enterprise clarity** — information first; chrome second.
2. **Operational speed** — the daily user's primary action is always one click away.
3. **Role-first experience** — every screen answers "what do *I* need to do now?"
4. **Calm premium identity** — restrained NAFFCO red as accent, generous whitespace, subtle elevation; never flashy.
5. **Action visibility** — approvals, gates, and blockers are impossible to miss.

6. **No fake confidence** — never show mock numbers as if live; label dev/mock states.
7. **Workflow safety** — UI never lets a user bypass a governance gate.
8. **Data density with readability** — `tabular-nums`, tracked headers, comfortable row height.
9. **Progressive disclosure** — summary first, detail on demand (tabs, drawers).
10. **Consistent status language** — one status always renders one way, everywhere.

15. Future Design System Blueprint

15.1 Token architecture

- **Color:** semantic tokens layered over the brand/charcoal scales — `--color-primary` (= brand red), `--color-surface-1/2/3`, `--color-text-{primary,secondary,muted}`, status tokens `--color-status-{neutral,info,warning,success,critical}-{bg,fg,border}`.
- **Typography:** semantic scale — `display`, `h1-h4`, `body`, `body-sm`, `caption`, `mono` — mapped to size/line-height/weight.
- **Spacing:** keep Tailwind scale; document a 4px base and standard page paddings.
- **Radius:** `--radius` (6px) as the single source; ban `rounded-xl` at page level.
- **Shadows:** existing `xs-xl` scale; cards use `sm`, popovers `md`, modals `lg`.
- **Surfaces:** page = surface-1 (off-white), cards = surface-2 (white), nested = surface-3.

15.2 Component variants

- **Button:** `primary | secondary | ghost | danger` × `sm | md | lg`, loading + icon slots.
- **Badge/StatusBadge:** the five status tokens, driven by a central map.
- **Card:** padding scale + optional header slot.
- **Input/Select/Textarea:** `border-gray-200 bg-white`, AA focus ring, error state.

15.3 Standards

- **Layout:** `PageShell` = header + breadcrumb + content max-width; one `PageHeader`.
- **Page templates:** `ListPageTemplate`, `DetailPageTemplate` (tabbed), `FormPageTemplate`, `DashboardTemplate`, `ReportTemplate`.
- **Tables:** uppercase tracked headers, `tabular-nums`, skeleton loading, empty state, optional pagination.
- **Forms:** `FormSection` + `FormField` (label, control, help, error) on RHF + zod.
- **Empty/Loading/Error:** one each — `EmptyState`, skeletons, `ErrorState`.
- **Dashboards/KPIs:** one `KpiCard` (label uppercase, value `tabular-nums`, optional trend/href).

15.4 Documentation

- **Storybook** for every `ui/` and `patterns/` component (stories + prop docs).
- A short **design-system MD** describing tokens, variants, and governance rules.
- A **design-review checklist** in PR templates.

16. A-to-Z Execution Roadmap

Each phase lists objective · scope · likely files · steps · risks · protected logic · validation · outcome · effort · dependencies · go/no-go.

Phase 0 — Stabilize & protect

- **Objective:** clean baseline before transformation.
- **Scope:** burn down 74 lint problems; freeze a token contract doc; add lint rules banning legacy classes (warn-only first).
- **Files:** lint config, `index.css` / `tailwind.config.js` (no value changes), targeted page lint fixes.
- **Steps:** fix by rule (`no-explicit-any`, `set-state-in-effect`, `no-empty-object-type`); document tokens.
- **Risks:** low. **Protected:** all workflows. **Validation:** `tsc`, `eslint` → 0 errors. **Outcome:** green CI. **Effort:** M. **Deps:** none. **Go/no-go:** proceed once lint errors = 0.

Phase 1 — Finish visual UX rollout by module

- **Objective:** bring remaining ~7 modules to the 19.x standard.
- **Scope:** Store, Factory, QC, AFS, After-Sales, Admin, Viewer, Reports (visual-only).
- **Files:** module page files only.
- **Steps:** apply the proven recipe (skeletons, `rounded-lg`, tracked headers, `tabular-nums`, `bg-white` inputs, `EmptyState`); one module per PR.
- **Risks:** low-med (QC/Store touch gates — visual only). **Protected:** receiving/custody/serials, QC release, AFS readiness. **Validation:** `tsc`, `eslint`, changed-file lint, manual smoke. **Outcome:** consistent look across all modules. **Effort:** L. **Deps:** Phase 0. **Go/no-go:** each module PR independently mergeable.

Phase 2 — Component consolidation & DS governance

- **Objective:** one component per concept; enforce it.
- **Scope:** merge duplicate Button/Card/Badge/PageHeader/EmptyState/loading; create `patterns/`; central status map; semantic color tokens; split `navigation.ts`.
- **Files:** `components/**`, call sites (94 Button, 105 page-header), `roleMatrix` / nav data (structure only).
- **Steps:** define canonical APIs; migrate call sites; delete deprecated layers; add lint bans (error level).
- **Risks:** med (wide call-site churn). **Protected:** no logic change. **Validation:** `tsc`, full `eslint`, visual diff on top pages. **Outcome:** single DS. **Effort:** L. **Deps:** Phase 1. **Go/no-go:** zero imports of deprecated components before deletion.

Phase 3 — Page architecture refactor

- **Objective:** tame mega-pages.
- **Scope:** split ProjectDetail, QuotationDetail, PODetail, WoPnGate, FactoryWorkspace, AdminApprovals into co-located tab/section sub-components.
- **Files:** the listed pages + new sibling component files.
- **Steps:** extract tab bodies; keep all hooks/logic in the container; no behavior change.
- **Risks:** med-high (gate pages). **Protected:** SO approval/routing, WO/PN gate, PO approval, QC release. **Validation:** `tsc`, `eslint`, full manual workflow walkthrough per page, \$19 checklist. **Outcome:** maintainable pages. **Effort:** L. **Deps:** Phase 2. **Go/no-go:** workflow walkthrough passes before merge.

Phase 4 — Data-loading / React Query foundation

- **Objective:** one query/cache layer.
- **Scope:** add `@tanstack/react-query` ; create `src/lib/queries/` hooks per domain; migrate list pages first, then detail tabs.
- **Files:** new query hooks; page fetch sections.
- **Steps:** wrap app in `QueryClientProvider` ; convert page-by-page, preserving mock-vs-Supabase branching and query meaning exactly.
- **Risks:** med. **Protected:** query semantics/filters unchanged. **Validation:** `tsc` , `eslint` (eliminates `set-state-in-effect`), data parity checks. **Outcome:** caching, dedup, no effect-bug class. **Effort:** L. **Deps:** Phase 0; parallelisable with 1–3. **Go/no-go:** data parity verified per page.

Phase 5 — Performance, pagination, lazy loading

- **Objective:** scale + speed.
- **Scope:** server-side pagination on list pages; per-tab fetching on detail pages; lazy-load heavy report/chart bundles.
- **Files:** list pages, detail containers, report pages.
- **Steps:** add `.range()` pagination via query hooks; split detail fetches per active tab.
- **Risks:** med (query meaning). **Protected:** filters/ordering. **Validation:** parity + perf check. **Outcome:** fast at scale. **Effort:** M-L. **Deps:** Phase 4.

Phase 6 — Reports & executive dashboards

- **Objective:** standardise + de-mock reports.
- **Scope:** `ReportTemplate` , `KpiCard` , real queries, print polish; Control Tower dashboard.
- **Files:** 15 report pages, Control Tower.
- **Steps:** template extraction; replace mock arrays with cached queries; unify print CSS.
- **Risks:** med. **Protected:** export logic. **Validation:** print + data checks. **Outcome:** leadership-grade reporting. **Effort:** L. **Deps:** Phases 2,4.

Phase 7 — Accessibility & responsive polish

- **Objective:** AA + mobile.
- **Scope:** contrast fixes, focus-visible, keyboard-reachable rows, responsive tables/forms.
- **Files:** DS components + pages.
- **Steps:** a11y audit on top 20 pages; fix patterns centrally.
- **Risks:** low. **Validation:** keyboard + screen-reader pass; automated a11y lint. **Outcome:** inclusive UX. **Effort:** M. **Deps:** Phase 2.

Phase 8 — Testing, QA & release governance

- **Objective:** safety net.
- **Scope:** Vitest + React Testing Library for DS components and critical flows; Playwright smoke for the governance gates; PR template + design-review checklist.
- **Files:** new test files, CI config.
- **Steps:** test DS components first, then gate workflows.
- **Risks:** low. **Validation:** CI green with tests. **Outcome:** cheap, safe change. **Effort:** L. **Deps:** Phase 2.

Phase 9 — Optional UI library migration *(decision gate — recommended: NO)*

- **Objective:** revisit only if Option C proves insufficient.
 - **Go/no-go:** proceed **only** if, after Phases 0–8, the internal DS still cannot meet a concrete need. **Default decision:** do not migrate.
-

17. PR-by-PR Implementation Plan

#	PR TITLE	OBJECTIVE	SCOPE	FILES	FORBIDDEN	VALIDATION	MERGE CRITERIA	ROLLBACK RISK	PRIORITY
1	Phase 0 — lint burndown (no-explicit-any)	Remove any debt	Type the 23 sites	affected files	logic change	tsc+eslint	0 errors in scope	Low	P0
2	Phase 0 — fix set-state-in-effect	Remove effect-bug class	17 sites	affected pages	query meaning	tsc+eslint	rule clean	Low	P0
3	Phase 0 — token contract doc + warn-lint	Freeze tokens	docs + lint cfg	docs, eslint	value changes	eslint warns	doc merged	Low	P0
4	Step 19.5B — Store UX	Modernise Store	12 pages	writes/custody/serials	logic	tsc+eslint+manual	smoke pass	Low	P1
5	Step 19.6 — Factory UX	Modernise Factory	8 pages	WO gate	logic	as above	smoke pass	Low	P1
6	Step 19.7 — QC UX	Modernise QC	3 pages	NCR/release gate	logic	heavy manual	gate walkthrough	Med	P1
7	Step 19.8 — AFS UX	Modernise AFS	6 pages	readiness	logic	smoke	pass	Low	P1
8	Step 19.9 — After-Sales + Viewer + Admin UX	Modernise low-risk	~16 pages	role guards	logic	smoke	pass	Low	P1
9	Step 19.10 — Reports UX pass (visual)	Modernise reports	15 pages	export logic	logic	print check	pass	Low	P1
10	DS-Consolidate Button	One Button	ui/Button , 94 call sites	API drift	tsc+eslint+visual	0 legacy imports	Med	P2	P2
11	DS-Consolidate Card/Badge/PageHeader/EmptyState	One each	components + call sites	API drift	as above	0 legacy imports	Med	P2	
12	DS-StatusBadge + central map	Consistent status	patterns/ + usages	status meaning	visual	pass	Low	P2	
13	Split ProjectDetail	Tame mega-page	ProjectDetail + subs	approval/gate logic	full walkthrough	\$19 checklist	High	P3	
14	React Query foundation + list migration	Cache layer	provider + list hooks	query meaning	parity	parity pass	Med	P4	

18. Recommended First 10 Implementation Prompts (high level)

- "Phase 0 lint burndown — no-explicit-any ": type the 23 any sites in changed files only; no logic change. *Why first: clean baseline, zero risk.*
- "Phase 0 — eliminate set-state-in-effect (17 sites)" using the established init-state / Promise.resolve().then() pattern. *Why: removes the dominant bug class pre-refactor.*
- "Token contract doc + warn-level lint rules" banning rounded-xl / legacy headers / border-gray-300 . *Why: stops new drift while rollout proceeds.*
- "Step 19.5B — Store / Warehouse UX" (visual-only, protect receiving/custody/serials). *Why: highest module priority (score 27).*
- "Step 19.6 — Factory UX" (protect WO gate). *Why: high ops value.*
- "Step 19.7 — QC UX" (protect NCR/release gate, heavy validation). *Why: governance-critical surface.*
- "Step 19.8/19.9 — AFS + After-Sales + Viewer + Admin UX" batched low-risk modules. *Why: fast consistency wins.*
- "DS consolidation — single Button" migrate 94 call sites, delete the loser. *Why: highest-leverage component cleanup.*
- "DS consolidation — Card/Badge/PageHeader/EmptyState + StatusBadge". *Why: completes the single-layer goal.*
- "React Query foundation + migrate list pages" preserving query meaning and mock branching. *Why: unlocks caching/perf and kills the effect-bug class permanently.*

19. Risk Management Plan

RISK	LIKELIHOOD	IMPACT	MITIGATION
Design regression	Med	Med	One module per PR; visual smoke; warn-then-error lint
Business-logic regression	Low	High	Visual-only PRs in Phase 1; \$19 gate checklist; never move mutations/validation
Route/access regression	Low	High	Never touch App.tsx /guards/ roleMatrix in UX PRs; separate any nav PR
Data-migration risk	Low	High	No DB/RLS/migration changes in this program; query meaning preserved in Phase 4
RLS/security risk	Low	High	No Supabase policy or storage permission changes; document-upload behavior frozen
Performance regression	Low	Med	React Query added behind parity checks; measure before/after
Delivery risk	Med	Med	Independent, mergeable PRs; phases parallelisable (4 alongside 1-3)
User-adoption risk	Low	Med	Visual continuity (evolve not rewrite); pages stay recognisable

Gate-protection checklist (apply to any PR touching ProjectDetail, WoPnGate, PODetail, QC release, Store custody/serials, AFS readiness): confirm no change to mutation payloads, validation rules, status transitions, approval/routing, role checks, RLS/storage; walk the full workflow manually in dev mode; run tsc + eslint + changed-file lint; get a second review.

20. Final Recommendation

- **Keep or replace the stack? Keep and formalise.** Build Option C (internal NAFFCO design system) on the existing Radix + Tailwind + CVA foundation. Do **not** migrate to MUI/Ant/Mantine.
- **Consolidate components first or finish page UX first? Finish the module UX rollout first (Phase 1), then consolidate (Phase 2).** The rollout recipe is proven, low-risk, and delivers visible value immediately; consolidation is cleaner once every module is on the same visual language. Run Phase 0 before both.
- **React Query now or later? Soon, but as its own foundation (Phase 4), and it can run in parallel with the module rollout.** It is the highest-value structural fix after consolidation and permanently removes the `set-state-in-effect` class.
- **Refactor ProjectDetail now or later? Later (Phase 3), after the design system is consolidated** — split it into co-located tab components with the gate checklist applied. It is the riskiest page; do it deliberately, not opportunistically.
- **Next immediate step: Phase 0 lint burndown**, then **Step 19.5B — Store / Warehouse UX** (highest module priority).
- **Safest path to world-class UX:** evolve in small, independently mergeable, visual-first PRs; protect every governance gate; consolidate to one design system; add a query layer and a test net — never a big-bang rewrite.

Appendix A — Evidence Index (measured at SHA 1ed145a)

METRIC	VALUE
Page files / total lines	124 / 41,938
Largest page	<code>ProjectDetail.tsx</code> (2,081)
Pages > 700 lines	8
Routes / lazy-loaded	125 / 124
Roles	10
Nav labels / nav file size	145 / 1,089 lines
Pages using manual <code>useEffect</code>	103
Pages referencing <code>supabase</code>	112
React Query / SWR	none
<code>react-hook-form</code> / <code>zod</code> page usage	0 / 0
<code>@tanstack/react-table</code> usage	1
Unit tests / e2e specs	0 / 1
ESLint problems (errors/warnings)	74 (38 / 36)
Pages with legacy <code>rounded-xl</code>	77
Pages with legacy table headers	48
Duplicate core components	Button, Card, Badge, PageHeader, EmptyState, loading
Custom Button page imports	94
<code>common/page-header</code> imports	105

Appendix B — PDF Generation

This report was rendered to PDF using the Chromium binary bundled with the repository's Playwright install (`/opt/pw-browsers/chromium-1194`), via the print-ready HTML companion. To regenerate:

```
# from repo root
node docs/reports/_render-pdf.cjs # if the helper is retained
# or open the .html file and "Print → Save as PDF" (A4, default margins)
```

End of report.